EE5203 L07 Lab Guide

Timer interrupts — a 1 ms tick instead of `HAL_Delay()` - Basri Erdogan

Mission: Build a 1 ms tick from a TIM2 update interrupt, drive a heartbeat blink from it with no `HAL_Delay()` anywhere, and prove in the SFR view that the hardware counts while your loop stays free.

Prepare before class

- ☐ Re-read the theory slides up to "A 1 ms tick counter from the timer."
- ☐ Compute from 16 MHz: which `PSC`/`ARR` give a 1 ms update event?
- ☐ Know which register bit records the update event and who clears it.

Learning objectives

By checkoff time, you can:

1. configure a TIM2 time base in CubeMX from the real 16 MHz clock;
2. run an update-interrupt callback and keep it short;
3. schedule LED phases from a millisecond counter, wrap-safe;
4. drive a bare-metal microsecond timer and explain every register write.

Hardware and files

Item	Required value
Board	Nucleo-F401RE — onboard LD2 (PA5) only, **no wiring**
Clock	**HSI, PLL off** (you set this in CubeMX, step 1 below): `SYSCLK` = `PCLK1` = timer clock = **16 MHz**
Project	New CubeMX project `L07_Timer_Tick` (Toolchain = CMake), opened in VS Code
Timer	TIM2, internal clock source

Configure in CubeMX

1. **Clock Configuration tab** (next to Pinout): set **System Clock Mux** to **HSI** and the **AHB**, **APB1** and **APB2** prescalers to **/1**. Every box on the right now reads **16**, including **APB1 Timer clocks (MHz)**. The board default runs the chip at 84 MHz through the PLL; with that setting every time in this lab is 5.25 times too short.
2. `PA5` → `GPIO_Output` (defaults: push-pull, no pull, low speed).
3. **Timers → TIM2**: Clock Source = `Internal Clock`; Prescaler `15` (16 MHz ÷ 16 = 1 MHz), Counter Period `999` (1 MHz ÷ 1000 → 1 ms).
4. **System Core → NVIC**: enable **TIM2 global interrupt**.
5. Generate code, then **verify — do not add**: `MX_TIM2_Init()` in `main.c` and `TIM2_IRQHandler()` calling `HAL_TIM_IRQHandler(&htim2)` in `stm32f4xx_it.c`.

Checkpoint A - The tick runs

In `USER CODE BEGIN PV` and `USER CODE BEGIN 4`:

```
volatile uint32_t g_ms = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        g_ms++;
    }
}
```

In `USER CODE BEGIN 2`: `HAL_TIM_Base_Start_IT(&htim2);`

Evidence for Checkpoint A: a breakpoint in the callback is hit; `g_ms` grows between pauses; in the SFR view `TIM2->PSC` = 15, `TIM2->ARR` = 999, and `TIM2->CNT` is moving.

Checkpoint B - Heartbeat without delay

Blink a heartbeat — LD2 **on for 100 ms, off for 900 ms** — using only `g_ms`:

```
uint32_t t_phase = 0;
uint8_t  led_on  = 0;

while (1) {
    uint32_t wait = led_on ? 100U : 900U;
```

```
    if ((g_ms - t_phase) >= wait) {
        t_phase = g_ms;
        led_on = !led_on;
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5,
                          led_on ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
    /* the loop stays free for other work */
}
```

The unsigned subtraction stays correct when `g_ms` wraps — L06's debounce arithmetic again.

Evidence for Checkpoint B: the 100/900 pattern is visible and `main.c` contains no `HAL_Delay()` call.

Checkpoint C - Bare-metal microsecond timer

In a separate source file or `#ifdef` block, rebuild the blink at 250 ms on/off with **no HAL timer calls**:

```
RCC->APB1ENR |= (1U << 0);    // TIM2 clock (APB1, not AHB1!)
TIM2->PSC = 15;               // 16 MHz / 16 = 1 MHz: 1 tick = 1 us
TIM2->ARR = 0xFFFFFFFFU;      // free-run: TIM2 is 32-bit
TIM2->EGR = (1U << 0);        // force update: load PSC now
TIM2->CR1 |= (1U << 0);       // CEN: start counting

static inline void delay_us(uint32_t us)
{
    uint32_t start = TIM2->CNT;
    while ((TIM2->CNT - start) < us) { }
}
```

Evidence for Checkpoint C: the blink runs; you can explain the `EGR` write and why the subtraction survives counter wrap (~71 minutes at 1 MHz).

Individual extension - Button-selected speed

Using the Checkpoint B loop: B1 (PC13, input, **no pull** — the external resistor is on the board) cycles the blink period through 500 → 200 → 50 ms. Debounce with 50 ms of `g_ms`, exactly as in L06. Optional: add `__WFI();` at the loop end and confirm timing still holds.

Acceptance checklist

- ☐ `.ioc`: TIM2 PSC 15 / ARR 999, NVIC enabled; PA5 output.
- ☐ Callback checks `htim->Instance` and only increments a counter.
- ☐ SFR evidence: PSC, ARR, moving CNT shown for Checkpoint A.
- ☐ No `HAL_Delay()` anywhere in the HAL tasks.
- ☐ Bare-metal version explained register by register.
- ☐ Extension demonstrated with clean speed switching.

Individual oral check

- Why 15 and 999 — and what breaks without the two +1s?
- What would happen if the callback did the LED writes and took 2 ms?
- Why is `g_ms` declared `volatile`, and why is the subtraction wrap-safe?
- `TIM2` is on which bus, and which register enables its clock?

Submit

Submit `main.c` (plus the bare-metal file), and one SFR-view screenshot showing `TIM2->PSC`, `TIM2->ARR`, and a nonzero `TIM2->CNT`.

If you are stuck

Walk the pipeline left to right and fix the **first** silent stage: `PSC`/`ARR` values in the SFR view → `CNT` moving → `UIF` setting in `SR` → NVIC enabled → breakpoint in the callback → `g_ms` growing → your loop condition. **CNT frozen?** Timer never started or clock not enabled. **Callback never runs?** NVIC checkbox or `Start_IT` missing. **Everything about 5 times too fast** (the heartbeat, the tick)? The Clock Configuration tab is still on the PLL at 84 MHz: System Clock Mux = HSI, prescalers /1, generate code again. **Pattern timing drifts?** You reset `t_phase` to `g_ms` somewhere other than the phase change.